Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Фундаментальные науки»

КАФЕДРА «Прикладная математика»

Лабораторная работа № 7
по дисциплине «Типы и структуры данных»

Тема Сжатие по Хаффману

Студент Михайлов А.К.

Группа ФН12-31Б

Преподаватели Волкова Л.Л.

Москва, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **4**

1 Аналитическая часть **5**

1.1 Алгоритм Хаффмана 5

2 Конструкторская часть **6**

2.1 Структура Node 6

2.2 Структура Compare 6

2.3 Класс HuffmanCompressor 6

2.4 Класс NSynthParser 7

2.5 Класс CompressionResearch 7

3 Технологическая часть **9**

3.1 Реализация и выбор средств 9

3.2 Тестирование программы 9

3.3 Сравнение с результатом работы программы 13

4 Исследовательская часть **15**

4.1 Анализ коэффициента сжатия 15

ЗАКЛЮЧЕНИЕ **17**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **18**

ПРИЛОЖЕНИЕ А **19**

ВВЕДЕНИЕ

В данной лабораторной работе рассматривается алгоритм сжатия данных по Хаффману, лежащий в основе многих современных архиваторов.

Целью работы является описание и реализация алгоритма сжатия данных с помощью метода Хаффмана. Осуществление поставленной цели требует выполнения следующих задач:

1) описать алгоритм сжатия данных по Хаффману;
2) разработать описанный алгоритм;
3) реализовать его;
4) провести тестирование полученной реализации;
5) провести оценку эффекта от сжатия при работе с файлами разных размеров.

Задание индивидуального варианта: описать содержания исходных файлов нотной записи, взять столбец $instrument_family$ из файла $Test$ из открытого источника [1].

1 Аналитическая часть

1.1 Алгоритм Хаффмана

Алгоритм Хаффмана — это жадный алгоритм оптимального префиксного кодирования, который используется для сжатия данных. Он позволяет минимизировать среднюю длину кодируемого сообщения за счёт построения префиксного кода минимальной стоимости [2].

Алгоритм Хаффмана основан на построении бинарного дерева кодирования способом, описанным ниже.

1) Для каждого символа входного алфавита создаётся листовой узел дерева с весом, равным частоте появления символа.
2) Пока в дереве более одного узла:
   — выбираются два узла с наименьшими весами;
   — создаётся новый узел, являющийся их родителем, с весом, равным сумме весов этих двух узлов;
   — новый узел добавляется в множество узлов дерева.
3) Процесс повторяется до тех пор, пока не останется единственный корневой узел.

После построения дерева Хаффмана каждому символу присваивается уникальный код, который определяется путём обхода от корня к соответствующему листу: при движении по левой ветви к коду добавляется 0, по правой — 1. Такой подход гарантирует, что результирующие коды удовлетворяют префиксному условию (ни один код не является префиксом другого), что обеспечивает возможность однозначного декодирования. Поскольку основная вычислительная нагрузка алгоритма приходится на построение дерева на основе частот символов, его асимптотическая сложность составляет $O(n \log n)$, где n — количество уникальных символов во входных данных.

Вывод

Были описан алгоритм Хаффмана.

2 Конструкторская часть

2.1 Структура Node

Структура `Node` представляет собой узел дерева Хаффмана и содержит следующие поля:

— $value$ — числовое значение символа;

— $freq$ — частота встречаемости в данных;

— $left$ — указатель на левого потомка (для бита 0);

— $right$ — указатель на правого потомка (для бита 1).

2.2 Структура Compare

Структура `Compare` переопределяет оператор сравнения для организации приоритетной очереди, обеспечивая сортировку узлов по частоте встречаемости символов, чтобы в очереди первыми были узлы с наименьшей частотой.

2.3 Класс HuffmanCompressor

Класс `HuffmanCompressor` содержит реализации алгоритмов сжатия и распаковки данных по Хаффману. Ниже приведено описание основных методов данного класса.

Сжатие данных

Метод `compress` реализует алгоритм сжататия вектора целых чисел, используя алгоритм Хаффмана:

— подсчитывает частоты встречаемости каждого символа;

— строит приоритетную очередь из узлов;

— строит дерево Хаффмана, объединяя узлы с наименьшими частотами;

— генерирует коды Хаффмана обходом дерева (0 — влево, 1 — вправо);

— кодирует исходные данные полученными кодами.

Распаковка данных

Метод `decompress` восстанавливает исходные данные из сжатой битовой строки:

— начинает с корня дерева;

— читает биты сжатых данных по одному;

— переходит по дереву: 0 — влево, 1 — вправо;

— при достижении листа добавляет символ в результат;

— возвращается к корню для декодирования следующего символа.

Сохранение в файл

Метод `saveCompressed` сохраняет сжатые данные и дерево Хаффмана в бинарный файл:

— записывает количество уникальных символов);

— сохраняет таблицу частот в формате пары (символ, количество повторений);

— преобразует битовую строку в байты с дополнением нулями до кратного 8;

— записывает размер сжатых данных и сами данные;

— вычисляет общий размер файла.

Загрузка из файла

Метод `loadCompressed` загружает и распаковывает данные из бинарного файла:

— читает количество уникальных символов;

— восстанавливает таблицу частот из пар (символ, количество повторений);

— строит дерево Хаффмана на основе таблицы частот;

— читает данные;

— преобразует байты обратно в битовую строку;

— вычисляет и удаляет неиспользованные биты;

— декодирует битовую строку используя восстановленное дерево Хаффмана.

2.4 Класс NSynthParser

Вспомогательный класс `NSynthParser` отвечает за извлечение данных $instrument_family$ из $JSON$ файла $NSynth$ согласно индивидуальному варианту. Единственный метод данного класса `parseInstrumentFamilies` извлекает числовые значения $instrument_family$ для их последующего сжатия:

— открывает $JSON$ файл для чтения;

— ищет строки, содержащие $instrument_family$;

— извлекает числовые значения после двоеточия;

— добавляет значения в результирующий вектор;

— возвращает вектор из 4096 значений $instrument_family$.

2.5 Класс CompressionResearch

Класс `CompressionResearch` отвечает за проведение исследования эффективности сжатия. Ниже приведено описание основных методов данного класса.

Создание тестовых данных

Метод `createTestFiles` создает набор тестовых файлов разных размеров:

— берет базовый набор данных;

— для каждого из 10 файлов увеличивает размер на 50%;
— заполняет данные повторением базового набора;
— возвращает вектор тестовых файлов разного размера.

Исследование сжатия

Метод `conductResearch` анализирует эффективность сжатия тестовых данных:
— для каждого тестового файла вычисляет исходный размер;
— сжимает данные алгоритмом Хаффмана;
— вычисляет размер сжатых данных в байтах;
— рассчитывает коэффициент сжатия r, сравнивая размеры исходного и сжатого файлов по формуле:

$$r = \frac{S_{\text{исх}}}{S} \tag{2.1}$$

где $S_{\text{исх}}$ — размер исходного файла, S — размер сжатого файла;
— выводит результаты в виде таблицы.

Вывод

Была выполнена разработка основных структур и классов данных и алгоритмов для кодирования и декодирования методом Хаффмана.

3 Технологическая часть

3.1 Реализация и выбор средств

Для программной реализации алгоритмов использовалась среда разработки $Visual\ Studio\ Code$ [3]. Язык программирования, на котором была выполнена реализация алгоритмов — $C++$.

В листинге А.1 представлена программная реализация описанных алгоритмов.

3.2 Тестирование программы

Для проверки корректности работы алгоритма было проведено сравнительное тестиролвание на ожидаемый/полученный результат. Для этого был выбран $JSON$ файл $test.json$, содержащий 50 значений параметра, из которых 10 уникальных. $instrument_family$. На этих значениях вручную будет применен алгоритм кодирования Хаффмана 1.1, включающий в себя следующие этапы:

1) подсчёт частот встречаемости каждого уникального значения в исходном файле;
2) построение канонического дерева Хаффмана на основе полученных частот;
3) формирование таблицы кодов (сопоставление исходных значений с битовыми последовательностями);
4) непосредственная замена исходных данных на соответствующие коды и упаковка результирующей битовой последовательности в байты.

Результат ручного кодирования — полученный сжатый бинарный файл и таблица кодов — будет выступать в качестве эталона. Далее он будет подвергнут сравнительному анализу с выходным файлом, сгенерированным программой. Совпадение размера, содержимого сжатого файла и структуры кодов будет свидетельствовать о корректной реализации алгоритма.

Содержимое выбранного файла $test.json$ представлено в листинге 3.1.

```
{
  "examples": [
  {"instrument_family": 0},
  {"instrument_family": 0},
  {"instrument_family": 0},
  {"instrument_family": 0},
  {"instrument_family": 0},
  {"instrument_family": 0},
  {"instrument_family": 0},
  {"instrument_family": 0},
  {"instrument_family": 1},
  {"instrument_family": 1},
  {"instrument_family": 1},
  {"instrument_family": 1},
```

```
  {"instrument_family": 1},
  {"instrument_family": 1},
  {"instrument_family": 2},
  {"instrument_family": 2},
  {"instrument_family": 2},
  {"instrument_family": 2},
  {"instrument_family": 2},
  {"instrument_family": 3},
  {"instrument_family": 3},
  {"instrument_family": 3},
  {"instrument_family": 3},
  {"instrument_family": 4},
  {"instrument_family": 4},
  {"instrument_family": 4},
  {"instrument_family": 5},
  {"instrument_family": 5},
  {"instrument_family": 6},
  {"instrument_family": 6},
  {"instrument_family": 7},
  {"instrument_family": 7},
  {"instrument_family": 8},
  {"instrument_family": 9},
  {"instrument_family": 0},
  {"instrument_family": 0},
  {"instrument_family": 0},
  {"instrument_family": 0},
  {"instrument_family": 0},
  {"instrument_family": 1},
  {"instrument_family": 1},
  {"instrument_family": 1},
  {"instrument_family": 2},
  {"instrument_family": 2},
  {"instrument_family": 3},
  {"instrument_family": 4},
  {"instrument_family": 5},
  {"instrument_family": 6},
  {"instrument_family": 7},
  {"instrument_family": 8}
  ]
}
```

Листинг 3.1 — Содержимое файла $test.json$

Ожидаемый результат

Результаты подсчета количества повторений уникальных значений параметра в выбранном файле представлено в таблице 3.1.

Таблица 3.1 — Количество уникальных значений

Значение	Количество повторений
0	13
1	9
2	7
3	5
4	4
5	3
6	3
7	3
8	2
9	1

Графическое представление дерева Хаффмана приведено на рисунке 3.1.

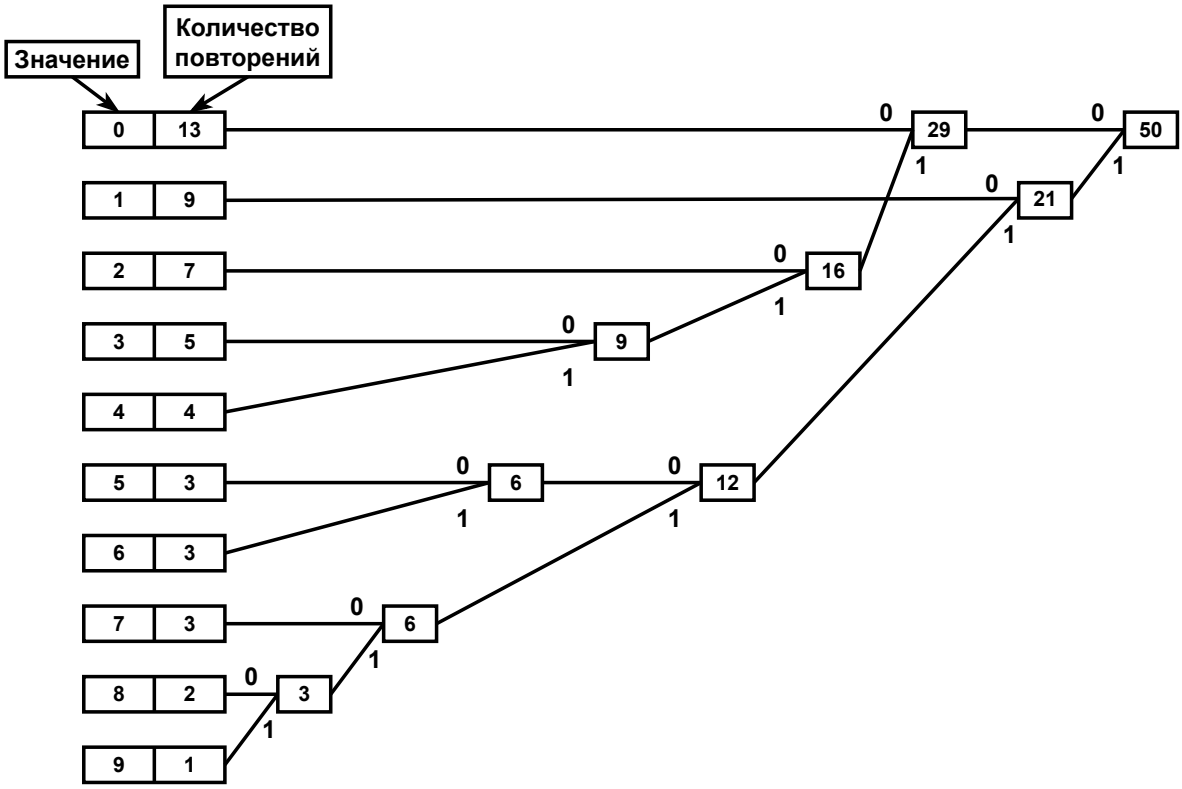

Рисунок 3.1 — Дерево Хаффмана

Полная таблица кодов Хаффмана значений параметра представлена в таблице 3.2.

Таблица 3.2 — Количество уникальных значений

Значение	**Код Хаффмана**	**Длина кода в битах**
0	00	2
1	10	2
2	010	3
3	0110	4
4	0111	4
5	1100	4
6	1101	4
7	1110	4
8	11110	5
9	11111	5

Размер S бинарного файла вычисляется по следующей формуле:

$$S = S_{\text{уник}} + S_{\text{табл}} + S_{\text{посл}} + S_{\text{знач}}, \tag{3.1}$$

где $S_{\text{уник}} = 4$ — размер ячейки памяти для хранения количества уникальных символов, $S_{\text{табл}}$ — размер таблицы повторений, $S_{\text{посл}} = 4$ — размер сжимаемых данных, $S_{\text{знач}}$ — размер сжатых данных. Все размеры указаны в байтах.

Формула для расчёта размера таблицы повторений:

$$S_{\text{табл}} = N \times S_{\text{int}} \times 2, \tag{3.2}$$

где $N = 10$ — количество уникальных значений, $S_{\text{int}} = 4$ байта — размер типа `int` в языке программирования $C++$, с помощью которого была выполнена реализация алгоритма Хаффмана.

$$S_{\text{табл}} = 10 \times 4 \times 2 = 80 \text{ байт} \tag{3.3}$$

Формула для расчёта общего количества бит:

$$B_{\text{бит}} = \sum_{i=1}^{10} (f_i \times l_i), \tag{3.4}$$

где f_i — частота i-го символа, l_i — длина кода Хаффмана для i-го символа

В таблице 3.3 приведен результат рассчета количества бит, занимаемых каждым уникальным значением.

Таблица 3.3 — Количество бит, занимаемых уникальными значениями

Значение	**Количество бит**
0	26
1	18
2	21
3	20
4	16
5	12
6	12
7	12
8	10
9	5

Таким образом, $B_{\text{бит}} = 152$ бита.

Формула для расчёта количества байт, занимаемых сжатыми значениями:

$$S_{\text{знач}} = \left\lceil \frac{B_{\text{бит}}}{8} \right\rceil \tag{3.5}$$

$$S_{\text{знач}} = \left\lceil \frac{152}{8} \right\rceil = 19 \text{ байт} \tag{3.6}$$

Количество полезных битов $B_{\text{посл}}$ в последнем байте:

$$B_{\text{посл}} = B_{\text{бит}} \bmod 8 = 152 \bmod 8 = 0 \tag{3.7}$$

Согласно формуле 3.1, общий размер S файла равняется 99 байтам:

$$S = 4 + 80 + 4 + 19 = 107 \tag{3.8}$$

Исходный файл содержит всего 50 целочисленных значений параметра, тогда его размер $S_{\text{исх}}$ равен 50×4 байтам, так как размер типа `int` равен 4 байтам. По формуле 2.1 имеем:

$$r = \frac{200}{107} = 1,87, \tag{3.9}$$

где r — коэффициент сжатия.

3.3 Сравнение с результатом работы программы

В таблице 3.4 представлен бинарный сжатый файл, полученный после работы программы с файлом $test.json$.

Таблица 3.4 — Полученный бинарный файл

Байты	**Байт 1**	**Байт 2**	**Байт 3**	**Байт 4**	**Байт 5**	**Байт 6**	**Байт 7**	**Байт 8**
00-07	0A	00	00	00	09	00	00	00
08-0F	01	00	00	00	08	00	00	00
10-17	02	00	00	00	07	00	00	00
18-1F	03	00	00	00	06	00	00	00
20-27	03	00	00	00	05	00	00	00
28-2F	03	00	00	00	04	00	00	00
30-37	04	00	00	00	03	00	00	00
38-3F	05	00	00	00	02	00	00	00
40-47	07	00	00	00	01	00	00	00
48-4F	09	00	00	00	00	00	00	00
50-57	0D	00	00	00	13	00	00	00
58-5F	00	00	00	00	AA	AA	FF	FF
60-67	F6	DB	12	48	01	11	55	99
68-6E	EE	AA	BF	FB	10	8A	CF	

Вывод

Программа успешно прошла тестирование на работоспособность.

4 Исследовательская часть

В данном разделе проводится исследование эффективности алгоритма Хаффмана посредством анализа зависимости коэффициента сжатия от размера исходных файлов.

4.1 Анализ коэффициента сжатия

Исследование по измерению времени работы реализаций алгоритмов проводилось на ноутбуке Huawei Matebook d16 [4] с 64-разрядной операционной системой Windows 11 [5]. Характеристики ноутбука: 8-ядерный процессор Intel Core i5-12450H [4] x64 с тактовой частотой 2 ГГц, оперативная память DDR4 8 ГБ. Для компиляции кода использовался компилятор $g++$.

На рисунке 4.1 приведёна зависимость коэффициента сжатия от размера файла в байтах. Для исследования были выбраны десять различных файлов с целыми числами, где каждое число является значением параметра $instrument_family$.

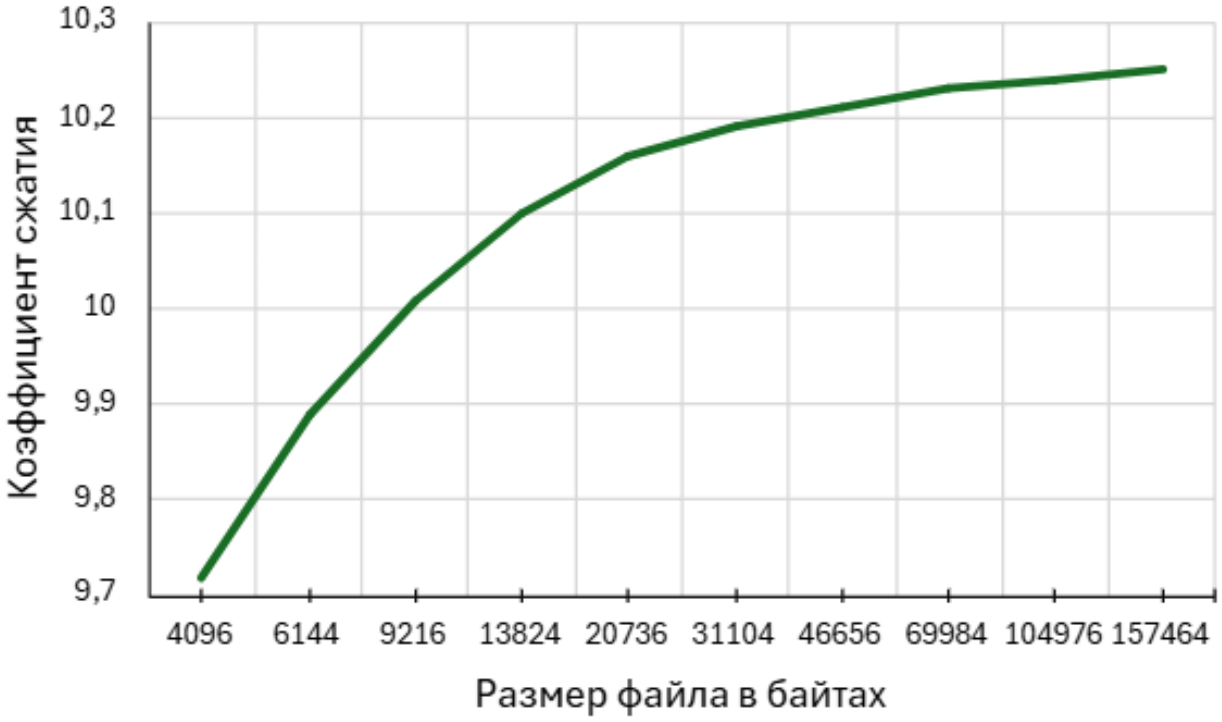

Рисунок 4.1 — Зависимость коэффициента сжатия от размера файлов

Анализ зависимости коэффициента сжатия от количества символов в файле показывает, что с увеличением объёма данных коэффициент сжатия возрастает. Он растет вместе с размером файла — с $9,72$ для маленьких файлов до $10,25$ для больших. Такое поведение объясняется особенностями алгоритма Хаффмана: при увеличении размера файла растёт эффективность кодирования, так как распределение частот символов становится более устойчивым, что позволяет лучше использовать возможности алгоритма для сокращения объёма данных. Однако скорость роста коэффициента сжатия уменьшается с увеличением количества символов, что указывает на приближение к предельной эффективности сжатия для выбранного набора данных.

Вывод

В данном разделе была проведена оценка степени сжатия по Хаффману при работе с 10 файлами разных размеров и был построен график зависимости коэффициента сжатия от размера исходного файла.

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторной работы была достигнута цель работы: был описан и реализован алгоритм сжатия данных по Хаффману.

Также были выполнены все задачи:

1) описан алгоритм сжатия данных по Хаффману;
2) описанный алгоритм разработан;
3) описанный алгоритм реализован;
4) проведено тестирование полученной реализации;
5) проведена оценка эффекта от сжатия при работе с файлами разных размеров.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Nsynth Dataset. [Электронный ресурс]. URL: https://magenta.tensorflow.org/datasets/nsynth#example-features (дата обращения: 22 ноября 2025).

2. Виноградова М.С. Сжатие данных. Алгоритм Хаффмана. // Журнал "Modern European Researches". — 2022.

3. Visual Studio Code documentation. [Электронный ресурс]. URL: https://code.visualstudio.com/docs (дата обращения: 21 ноября 2025).

4. Huawei. [Электронный ресурс]. URL: https://consumer.huawei.com/ru/laptops/matebook-14-2024/specs/?ysclid=mg5k19xzko358223160 (дата обращения: 21 ноября 2025).

5. Microsoft Ignite. [Электронный ресурс]. URL: https://learn.microsoft.com/ru-ru/windows (дата обращения: 21 ноября 2025).

6. NanoReview. [Электронный ресурс]. URL: https://nanoreview.net/ru/cpu/intel-core-i5-12450h (дата обращения: 21 ноября 2025).

ПРИЛОЖЕНИЕ А

В листинге А.1 представлена программная реализация описанных алгоритмов.

```

#include <iostream>
#include <fstream>
#include <vector>
#include <queue>
#include <unordered_map>
#include <algorithm>
#include <iomanip>
#include <string>
#include <cstdint>
#include <cmath>

using namespace std;

struct Node {
  int value;
  int freq;
  Node* left;
  Node* right;

  Node(int v, int f) : value(v), freq(f), left(nullptr), right(nullptr) {}
};

struct Compare {
  bool operator()(Node* a, Node* b) {
    if (a->freq != b->freq)
    return a->freq > b->freq;
    return getMinValueInSubtree(a) > getMinValueInSubtree(b);
  }

  private:
  int getMinValueInSubtree(Node* node) {
    if (!node) return INT_MAX;
    if (!node->left && !node->right)
    return node->value;

    return min(getMinValueInSubtree(node->left), getMinValueInSubtree(node->
   right));
  }
};

class HuffmanCompressor {
  private:
  unordered_map<int, string> huffmanCodes;
```

```
unordered_map<int, int> frequencies;
Node* root;

void buildCodes(Node* node, const string& code) {
  if (!node) return;

  if (!node->left && !node->right) {
    huffmanCodes[node->value] = code;
    return;
  }

  buildCodes(node->left, code + "0");
  buildCodes(node->right, code + "1");
}

void deleteTree(Node* node) {
  if (!node) return;
  deleteTree(node->left);
  deleteTree(node->right);
  delete node;
}

public:
HuffmanCompressor() : root(nullptr) {}

~HuffmanCompressor() {
  deleteTree(root);
}

string compress(const vector<int>& data) {
  if (data.empty())
  return "";

  frequencies.clear();
  for (int value : data)
  frequencies[value]++;

  priority_queue<Node*, vector<Node*>, Compare> pq;

  for (const auto& pair : frequencies)
  pq.push(new Node(pair.first, pair.second));

  while (pq.size() > 1) {
    Node* left = pq.top(); pq.pop();
    Node* right = pq.top(); pq.pop();

    int minValue = min(
```

```
      (left->left || left->right) ? INT_MAX : left->value,
      (right->left || right->right) ? INT_MAX : right->value
      );
      if (minValue == INT_MAX) minValue = -1;

      Node* parent = new Node(minValue, left->freq + right->freq);
      parent->left = left;
      parent->right = right;

      pq.push(parent);
    }

    if (!pq.empty())
    root = pq.top();
    else {
      root = nullptr;
      return "";
    }

    huffmanCodes.clear();
    buildCodes(root, "");

    string compressed;
    for (int value : data)
    compressed += huffmanCodes[value];

    return compressed;
  }

  vector<int> decompress(const string& compressedData) {
    vector<int> result;

    if (!root) {
      cerr << "Ошибка: дерево Хаффмана не построено!" << endl;
      return result;
    }

    Node* current = root;

    for (char bit : compressedData) {
      if (bit == '0')
      current = current->left;
      else if (bit == '1')
      current = current->right;

      if (!current->left && !current->right) {
        result.push_back(current->value);
```

```
        current = root;
      }
    }

    return result;
  }

  void saveCompressed(const string& filename, const string& compressedData)
   {
    ofstream file(filename, ios::binary);

    if (!file.is_open()) {
      cerr << "Ошибка создания файла: " << filename << endl;
      return;
    }

    int uniqueCount = frequencies.size();
    file.write(reinterpret_cast<const char*>(&uniqueCount), sizeof(
   uniqueCount));

    for (const auto& pair : frequencies) {
      file.write(reinterpret_cast<const char*>(&pair.first), sizeof(pair.
   first));
      file.write(reinterpret_cast<const char*>(&pair.second), sizeof(pair.
   second));
    }

    vector<unsigned char> bytes;
    string temp = compressedData;

    int padding = 8 - (temp.length() % 8);
    if (padding == 8) padding = 0;
    if (padding > 0)
    temp.append(padding, '0');

    for (int i = 0; i < temp.length(); i += 8) {
      string byteStr = temp.substr(i, 8);
      unsigned char byte = 0;
      for (int j = 0; j < 8; j++)
      if (byteStr[j] == '1')
      byte |= (1 << (7 - j));
      bytes.push_back(byte);
    }

    int dataSize = bytes.size();
    file.write(reinterpret_cast<const char*>(&dataSize), sizeof(dataSize));
    file.write(reinterpret_cast<const char*>(bytes.data()), bytes.size());
```

```

    file.close();

    int totalFileSize = 4 + (uniqueCount * 8) + 4 + dataSize;
    cout << "Сохранено: уникальных символов=" << uniqueCount
    << ", данные=" << dataSize << " байт"
    << ", общий размер файла=" << totalFileSize << " байт" << endl;
  }

  vector<int> loadCompressed(const string& filename) {
    ifstream file(filename, ios::binary);
    vector<int> result;

    if (!file.is_open()) {
      cerr << "Ошибка открытия файла: " << filename << endl;
      return result;
    }

    int uniqueCount;
    file.read(reinterpret_cast<char*>(&uniqueCount), sizeof(uniqueCount));

    frequencies.clear();
    for (int i = 0; i < uniqueCount; i++) {
      int symbol, freq;
      file.read(reinterpret_cast<char*>(&symbol), sizeof(symbol));
      file.read(reinterpret_cast<char*>(&freq), sizeof(freq));
      frequencies[symbol] = freq;
    }

    int dataSize;
    file.read(reinterpret_cast<char*>(&dataSize), sizeof(dataSize));

    vector<unsigned char> compressedBytes(dataSize);
    file.read(reinterpret_cast<char*>(compressedBytes.data()), dataSize);

    file.close();

    priority_queue<Node*, vector<Node*>, Compare> pq;

    for (const auto& pair : frequencies)
    pq.push(new Node(pair.first, pair.second));

    while (pq.size() > 1) {
      Node* left = pq.top(); pq.pop();
      Node* right = pq.top(); pq.pop();

      int minValue = min(
```

```
      (left->left || left->right) ? INT_MAX : left->value,
      (right->left || right->right) ? INT_MAX : right->value
      );
      if (minValue == INT_MAX) minValue = -1;

      Node* parent = new Node(minValue, left->freq + right->freq);
      parent->left = left;
      parent->right = right;

      pq.push(parent);
    }

    if (!pq.empty())
    root = pq.top();
    else {
      root = nullptr;
      return result;
    }

    huffmanCodes.clear();
    buildCodes(root, "");

    string bitString;
    for (unsigned char b : compressedBytes)
    for (int i = 7; i >= 0; i--)
    bitString += (b & (1 << i)) ? '1' : '0';

    int actualBits = dataSize * 8;
    int originalBits = 0;
    for (const auto& pair : frequencies)
    originalBits += pair.second * huffmanCodes[pair.first].length();
    int padding = actualBits - originalBits;

    if (padding > 0)
    bitString = bitString.substr(0, bitString.length() - padding);

    result = decompress(bitString);

    return result;
  }

  int calculateTotalCompressedSize(const string& compressedData) const {
    int uniqueCount = frequencies.size();
    int dataSize = (compressedData.length() + 7) / 8;
    return 4 + (uniqueCount * 8) + 4 + dataSize;
  }

```

```
  int calculateOriginalSize(const vector<int>& data) const {
    return data.size() * sizeof(int);
  }

  double calculateCompressionRatio(const vector<int>& originalData, const
   string& compressedData) const {
    double originalSize = calculateOriginalSize(originalData);
    double compressedSize = calculateTotalCompressedSize(compressedData);
    return originalSize / compressedSize;
  }

  void printHuffmanCodes() const {
    cout << "\n+---------------------------------------+" << endl;
    cout << "|         ТАБЛИЦА КОДОВ ХАФФМАНА         |" << endl;
    cout << "+------------+----------+--------------+" << endl;
    cout << "|   Символ   |  Частота |      Код     |" << endl;
    cout << "+------------+----------+--------------+" << endl;

    vector<pair<int, int>> sortedFreq;
    for (const auto& pair : frequencies)
    sortedFreq.push_back(pair);

    sort(sortedFreq.begin(), sortedFreq.end),
    [](const pair<int, int>& a, const pair<int, int>& b) {
      return a.second > b.second;
    });

    for (const auto& pair : sortedFreq) {
      int symbol = pair.first;
      int freq = pair.second;
      string code = huffmanCodes.at(symbol);

      cout << "| " << setw(10) << right << symbol
      << " | " << setw(8) << freq
      << " | " << setw(12) << code << " |" << endl;
    }
    cout << "+------------+----------+--------------+" << endl;
  }
};

class NSynthParser {
  public:
  static vector<int> parseInstrumentFamilies(const string& filename) {
    vector<int> families;
    ifstream file(filename);
    string line;

```

```
    if (!file.is_open()) {
      cerr << "Ошибка: Не удалось открыть файл " << filename << endl;
      return families;
    }

    while (getline(file, line)) {
      int pos = line.find("\"instrument_family\"");
      if (pos != string::npos) {
        int colonPos = line.find(":", pos);
        if (colonPos != string::npos) {
          string numberStr;
          for (int i = colonPos + 1; i < line.length(); i++) {
            if (isdigit(line[i]))
            numberStr += line[i];
            else if (!numberStr.empty())
            break;
          }

          if (!numberStr.empty()) {
            try {
              families.push_back(stoi(numberStr));
            } catch (...) {
              cerr << "Ошибка преобразования: " << numberStr << endl;
            }
          }
        }
      }
    }

    file.close();
    return families;
  }
};

class CompressionResearch {
  public:
  static vector<vector<int>> createTestFiles(const vector<int>& baseData,
   int count) {
    vector<vector<int>> testFiles;

    for (int i = 0; i < count; i++) {
      double multiplier = pow(1.5, i);
      size_t targetSize = static_cast<size_t>(baseData.size() * multiplier);

      vector<int> testData;
      while (testData.size() < targetSize) {
        size_t toCopy = min(baseData.size(), targetSize - testData.size());
```

```
        testData.insert(testData.end(), baseData.begin(), baseData.begin() + toCopy);
      }

      testFiles.push_back(testData);
    }

    return testFiles;
  }

  static void conductResearch(const vector<vector<int>>& testFiles) {
    cout << "\n=== ИССЛЕДОВАНИЕ СЖАТИЯ ===" << endl;
    cout << setw(16) << "Размер" << setw(25) << "Исходный" << setw(23) << "Сжатый"
    << setw(27) << "Коэффициент" << endl;
    cout << string(60, '-') << endl;

    HuffmanCompressor compressor;

    for (int i = 0; i < testFiles.size(); i++) {
      const auto& data = testFiles[i];
      double originalSize = data.size() * sizeof(int);
      string compressed = compressor.compress(data);
      double compressedSize = compressor.calculateTotalCompressedSize(compressed);
      double ratio = originalSize / compressedSize;

      cout << setw(10) << data.size()
      << setw(15) << fixed << setprecision(1) << originalSize << " 6"
      << setw(15) << fixed << setprecision(1) << compressedSize << " 6"
      << setw(15) << fixed << setprecision(2) << ratio << "x" << endl;
    }
  }
};

int main() {
  cout << "=== СЖАТИЕ ДАННЫХ NSynth АЛГОРИТМОМ ХАФФМАНА ===" << endl;

  cout << "\n1. Чтение данных из examples.json" << endl;
  vector<int> instrumentFamilies = NSynthParser::parseInstrumentFamilies("nsynth-test/examples.json");

  if (instrumentFamilies.empty()) {
    cerr << "Не удалось прочитать данные!" << endl;
    return 1;
  }

```

```
    cout << "Прочитано " << instrumentFamilies.size() << " значений instrument_family" << endl;

    HuffmanCompressor compressor;
    string compressed = compressor.compress(instrumentFamilies);

    double originalSize = compressor.calculateOriginalSize(instrumentFamilies);
    double totalCompressedSize = compressor.calculateTotalCompressedSize(compressed);
    double compressionRatio = compressor.calculateCompressionRatio(instrumentFamilies, compressed);

    cout << "Размер исходных данных: " << originalSize << " байт" << endl;
    cout << "Общий размер сжатого файла: " << totalCompressedSize << " байт" << endl;
    cout << "Коэффициент сжатия: " << fixed << setprecision(2) << compressionRatio << "x" << endl;

    compressor.printHuffmanCodes();
    compressor.saveCompressed("compressed_nsynth.bin", compressed);
    cout << "Сжатые данные сохранены в compressed_nsynth.bin" << endl;

    cout << "\n2. Проведение исследования" << endl;
    auto testFiles = CompressionResearch::createTestFiles(instrumentFamilies, 10);
    CompressionResearch::conductResearch(testFiles);

    return 0;
}
```

Листинг А.1 — Программная реализация разных алгоритмов умножения матриц